

Advanced Autonomous Navigation and Simulation Architectures for World of Warcraft 3.3.5a (Build 12340): A Comprehensive Technical Analysis

1. Introduction: The Engineering Landscape of MMORPG Automation

The development of autonomous agents for Massively Multiplayer Online Role-Playing Games (MMORPGs) represents a convergence of several high-level computer science disciplines: computational geometry, network protocol engineering, reverse engineering, and behavioral artificial intelligence. Specifically, the 3.3.5a version of World of Warcraft (WoW), corresponding to the *Wrath of the Lich King* expansion (Build 12340), remains a focal point for emulation research and automation development due to the stability of its client and the proliferation of open-source server cores like TrinityCore and AzerothCore.

This report provides an exhaustive technical analysis of the architectural requirements for building a robust, undetectable bot framework as an injected Dynamic Link Library (DLL). Unlike external "pixel bots" that rely on computer vision and keyboard event simulation, an injected framework operates within the game's virtual address space, allowing for direct memory access, function hooking, and seamless interaction with the game's internal rendering and logic loops. This internal access, while powerful, dramatically increases the surface area for detection by client-side protections such as Warden and server-side heuristics.

Therefore, the primary engineering challenge is not merely solving the pathfinding problem—moving from point A to point B—but doing so in a manner that is indistinguishable from human input. This requires a departure from standard "shortest-path" algorithms toward "plausible-path" generation, incorporating road preferences, organic curve smoothing, and reaction latency simulation. This document dissects the extraction of game geometry, the generation of navigation meshes (Navmeshes) using the Recast/Detour library, the implementation of weighted A* searches for road adherence, and the mathematical modeling of human mouse movement using spline interpolation. Furthermore, we analyze the movement validation logic of the server to define the operational constraints necessary to evade heuristic detection.

2. Bot Architecture: The Injected DLL Paradigm

The foundation of high-performance automation lies in its architecture. For complex tasks like

3D navigation and combat rotation, the latency introduced by inter-process communication (IPC) in external bots is often unacceptable. The Injected DLL model places the bot's code directly into the Wow.exe process, enabling synchronous execution with the game's main loop.

2.1 Process Injection and Lifecycle Management

The injection vector typically utilizes standard Windows API manipulation. The loader process obtains a handle to the game process via `OpenProcess` with `PROCESS_ALL_ACCESS` rights. Memory for the DLL path is allocated within the target process using `VirtualAllocEx`, and `WriteProcessMemory` populates this space. The actual loading is triggered by `CreateRemoteThread`, invoking `LoadLibraryA` with the allocated path as an argument.¹

However, for stealth, manual mapping is the superior technique. Instead of relying on the Windows loader (which leaves entries in the Process Environment Block's `LdrData` list, easily scanned by Warden), a manual mapper parses the DLL's headers, allocates memory for sections, resolves imports, and executes thread-local storage (TLS) callbacks manually. This results in a module that exists in memory but is invisible to standard API enumeration tools.²

Once resident, the bot must initialize its core subsystems: the Memory Manager, the Object Manager interface, the Navigation Engine, and the Script Engine. Critical to this initialization is the establishment of the "Main Thread Hook."

2.2 Synchronization via EndScene Hooking

World of Warcraft 3.3.5a utilizes DirectX 9 for rendering. The game loop operates by processing input, updating the simulation state, and finally rendering the frame. To ensure thread safety and data consistency, the bot's logic must execute at a deterministic point in this loop. The industry standard is to hook the `IDirect3DDevice9::EndScene` function.³

`EndScene` is virtual function index 42 in the `IDirect3DDevice9` Virtual Method Table (VMT). By detouring this function (replacing the pointer in the VMT or placing a `JMP` instruction at the function prologue), the bot intercepts the execution flow immediately after the game has finished submitting drawing commands but before the frame is presented to the screen.

The Execution Flow:

1. **Game Loop:** The client calculates physics, processes network packets, and issues draw calls.
2. **EndScene Call:** The client calls `EndScene` to signal the end of frame rendering.
3. **Detour:** Execution is redirected to `Bot::OnEndScene`.
4. **Bot Logic:**
 - o **State Update:** The bot reads the Object Manager to update the local cache of units and game objects. Because this happens on the main thread, the data is guaranteed to be stable; there is no risk of reading a pointer that is being deleted by the game engine simultaneously.

- **Decision Making:** The Finite State Machine (FSM) evaluates the current state (e.g., "In Combat", "Need Vendor") and determines the next action.
 - **Input Execution:** If movement or spell casting is required, the bot calls internal engine functions (e.g., ClickToMove, CastSpell) directly.
 - **Rendering:** The bot draws its own debug overlays (navmesh wireframes, path lines, status text) using the DirectX device pointer passed to EndScene.
5. **Return:** The bot calls the original EndScene function, allowing the game to finish the frame and present it to the user.

This architecture ensures that the bot effectively becomes part of the game engine, reacting with zero latency to game events.

2.3 The Finite State Machine (FSM)

The behavioral logic is typically structured as a Hierarchical Finite State Machine (HFSM). At the highest level, the bot operates in states such as Grind, Travel, Rest, or Vendor. Each state manages a subset of behaviors.

Example State: Travel

The Travel state is responsible for navigating from the current position to a target coordinate. It interacts heavily with the Navigation System.

- **Entry:** Calculate path from P_{start} to P_{end} using the Navmesh.
- **Update:**
 - Check distance to current waypoint.
 - If $Distance < Threshold$, increment waypoint index.
 - Calculate facing angle required to reach the next waypoint.
 - Apply movement input (simulated keypress or CTM packet).
 - **Interrupt:** If a hostile mob enters the aggro radius, transition to Combat state.
 - **Stuck Check:** If position has not changed significantly in T seconds despite movement input, trigger Unstuck routine (jump, strafe, or recalculate path).

3. Geometry Extraction: Parsing the World of Warcraft

Before a bot can navigate, it must understand the world. Unlike a human who sees obstacles visually, a bot requires a mathematical representation of the terrain. This necessitates extracting the raw geometry files from the game's MPQ archives and converting them into a format suitable for pathfinding algorithms. The primary file formats involved are ADT, WMO, and M2.

3.1 ADT (Area Definition Table) Analysis

The ADT files represent the terrain of the open world (Kalimdor, Eastern Kingdoms, Northrend).

The world is divided into a grid of map tiles, each covered by a specific ADT file (e.g., Azeroth_30_48.adt). A single ADT file covers a 533.333 yard square area.⁴

Internal Structure:

The ADT format is chunk-based, utilizing a RIFF-like structure. The vital chunks for navigation are:

- **MHDR (Header):** Contains offsets to other chunks.
- **MCVT (Map Chunk Vertices):** This chunk contains the heightmap data. Each map chunk (there are 16x16 chunks in one ADT) contains a 9x9 + 8x8 grid of height values. These values define the Z-coordinate (elevation) of the terrain grid.⁵
- **MCNR (Normals):** Contains normal vectors for lighting, useful for determining slope steepness, although Recast calculates this from geometry.
- **MCLY (Texture Layers):** This chunk defines which textures are painted on the terrain. It references texture filenames stored in the **MTEX** chunk.
- **MH2O (Water):** Added in WotLK, this chunk defines the water levels and types (ocean, river, slime). This is critical for assigning high travel costs to water or marking deep water as non-walkable.⁶

Extraction Logic: To build the navmesh geometry, the extractor must iterate through every ADT file. For each MCVT chunk, it generates a mesh grid. If the bot framework aims to implement road preference, the extraction phase is the critical point for data tagging. The extractor must correlate the texture layers defined in MCLY with the specific triangles in the generated mesh. If a texture corresponds to a road (e.g., Tileset\Northrend\Dragonblight\Road_Dirt_01.blp), the corresponding triangles in the mesh are flagged with a specific Area ID (e.g., AREA_ROAD).⁶

3.2 WMO (World Map Object) Integration

WMOs represent static, complex structures such as castles, caves, and ruins. Unlike the heightmap-based ADT, WMOs are arbitrary 3D meshes. They are referenced in the ADT via the **MODF** (Map Object Definition) chunk, which provides the position, rotation, and scale of the WMO instance.

Complexities in WMO Extraction:

- **Root vs. Group Files:** A WMO consists of a "Root" file (defining the skeleton and materials) and multiple "Group" files (containing the actual geometry for different parts of the model).⁸
- **Portals:** WMOs use a portal system for visibility culling. The geometry extraction must accurately process these to understand connectivity between indoor and outdoor spaces.
- **Liquid Data:** WMOs can contain their own liquid data (e.g., lava inside Blackrock Mountain), which must be extracted to prevent bots from walking into indoor hazards.

When generating the geometry for Recast, the WMO meshes must be transformed into world space using the coordinates from the ADT's MODF chunk and merged with the ADT terrain mesh. This ensures that the navmesh allows seamless transition from the terrain into a building.⁸

3.3 M2 Models (Doodads)

M2 files are smaller objects like trees, fences, and mailboxes. They are referenced in the ADT's **MDDF** (Map Doodad Definition) chunk.

Collision Geometry:

Detailed rendering meshes of M2s are too complex and unnecessary for navigation. Instead, most M2s contain simplified collision geometry (bounding boxes, cylinders, or low-poly meshes). The extraction tool must parse the M2 file to locate this collision data. If collision data is missing, a bounding box is often generated based on the model's extents.

Role in Pathfinding: M2s act as obstacles. In the Recast pipeline, they are rasterized as solid voxels, creating "holes" in the navmesh. This prevents the bot from trying to walk through a tree or getting stuck on a fence. Accurately extracting M2 collision data is vital for "clean" pathfinding; otherwise, the bot will constantly collide with invisible (to the navmesh) objects.¹¹

4. Navigation Mesh Generation: The Recast Pipeline

With the raw geometry extracted (often gigabytes of vertex data), the next step is to compile it into a queryable Navigation Mesh. The **Recast** library is the industry standard for this task and is used by both the client (implicitly) and server emulators like TrinityCore.¹²

4.1 The Voxelization Process

Recast does not work directly with triangles during the generation phase. Instead, it "rasterizes" the geometry into a 3D grid of voxels (Heightfield).

1. **Configuration:** The generation parameters must align with WoW's movement physics.
 - cs (Cell Size): 0.25 to 0.33 yards. This defines the resolution of the navmesh. A smaller value yields higher precision but increases memory usage and generation time.
 - ch (Cell Height): 0.3 yards.
 - walkableSlopeAngle: 50 degrees. WoW characters can climb relatively steep slopes. Setting this correctly is crucial to prevent the bot from getting stuck on hills that are visually walkable.¹³
 - walkableClimb: 0.5 to 1.0 yards. Defines the maximum height of a "step" (like a curb or stair).
 - walkableRadius: 0.6 yards. The radius of the character. Recast "erodes" the walkable area by this radius to ensuring that the *center* point of the agent never gets too close

to a wall.¹⁵

2. **Rasterization:** The input triangles (from ADT, WMO, M2) are rasterized into the heightfield. "Solid" spans are marked.
3. **Filtering:**
 - **Ledge Spans:** Removes areas where the drop-off is too steep.
 - **Low Height Spans:** Removes areas where the clearance between the floor and ceiling is less than the character's height (approx 2 yards).

4.2 Region Partitioning and Polygon Generation

Once the walkable surface is defined in voxels, Recast partitions the surface into regions. This is where the **Road Preference** logic is physically baked into the mesh.

- **Area Marking:** During rasterization, we use the texture data extracted earlier to mark specific spans with Area IDs.
 - AREA_ROAD (ID 1): Triangles corresponding to road textures.
 - AREA_GROUND (ID 2): Default terrain.
 - AREA_WATER (ID 3): Water surfaces (if swimming is supported) or shallow water.
- **Region Merging:** Recast groups connected spans with the same Area ID into polygonal regions.
- **Mesh Generation:** The boundaries of these regions are traced and simplified into convex polygons. This creates the final **Detour NavMesh**.

4.3 Tiled Navmesh Architecture

Generating a single navmesh for a continent like Kalimdor is computationally infeasible and would consume excessive memory. The solution is a **Tiled Navmesh**.

- **Grid System:** The world is split into a grid of tiles (typically matching the ADT grid, or subdivisions of it).
- **Streaming:** The bot framework maintains a cache of loaded tiles. As the player moves, tiles entering the "active radius" are loaded from disk (.mmtile files) and added to the dtNavMesh object, while distant tiles are removed.¹³
- **Linkage:** When a new tile is loaded, Detour automatically "stitches" it to adjacent loaded tiles, connecting the polygons at the edges to form a continuous graph.

5. Advanced Pathfinding: Implementing Road Preference

Standard pathfinding uses the A* algorithm to find the shortest path. However, a bot that strictly follows the shortest path (often a straight line across fields) looks unnatural and is easily flagged by heuristic analysis. To emulate human behavior, the bot should prefer roads even if the path is slightly longer.

5.1 Weighted Graphs with dtQueryFilter

The Detour library provides the dtQueryFilter class to customize the cost calculation of A*.

The cost function is defined as:

$$Cost(Path) = \sum_{i=0}^n Distance(node_i, node_{i+1}) \times AreaWeight(node_{i+1})$$

To implement road preference, we assign weights to the Area IDs defined during generation ¹⁶:

- **Road Weight:** 1.0 (Base cost).
- **Ground Weight:** 2.0 to 10.0 (Penalty cost).
- **Water Weight:** 20.0 (Severe penalty).

Scenario:

Consider a path from A to B. A straight line across a field (Ground) is 100 yards. A road connects them in an L-shape of 150 yards.

- **Direct Path Cost:** $100 \times 2.0 = 200$.
- **Road Path Cost:** $150 \times 1.0 = 150$.

The A* algorithm will minimize the total cost, selecting the road path despite the greater geometric distance. This simple weighting mechanism creates sophisticated, "human-like" routing behavior where the bot naturally seeks out roads, follows them, and only steps off-road when nearing the destination.¹⁸

5.2 Heuristic Tuning

The heuristic function $h(n)$ in A* estimates the remaining distance to the goal. For weighted pathfinding, the heuristic must be compatible with the weights. If the heuristic underestimates the cost too significantly (e.g., using straight Euclidean distance while Ground weight is 10.0), the A* search might expand too many nodes, degrading performance. Scaling the heuristic by the minimum weight (1.0) ensures the heuristic remains admissible (never overestimates), guaranteeing an optimal path relative to the weights.

6. Obstacle and Hostile Avoidance: The Tactical Layer

While the navmesh handles static geometry, the game world is dynamic. Hostile Non-Player Characters (NPCs) and dynamic objects must be avoided to ensure survival.

6.1 The Object Manager and Threat Assessment

The bot must periodically scan the Object Manager (every tick or few hundred milliseconds) to

identify threats.

- **Memory offsets (Build 12340):**
 - **Object Manager Base:** 0x00C79CE0 -> 0x2ED0.
 - **Entity Structure:** Iterating the linked list (NextObject at 0x3C) to read Unit types.
 - **Data retrieval:** Reading the FactionTemplate (to determine hostility), Level, and Position.

Threat Heatmap:

For every hostile unit detected, the bot calculates a "Threat Radius" (usually Aggro Radius + Safety Margin).

$$R_{threat} = R_{base_aggro} + (Level_{mob} - Level_{player}) \times C + Margin$$

6.2 Runtime Navmesh Modification

To avoid these threats, we cannot simply rely on reactive steering behaviors, as they might lead the bot into dead ends. We must update the pathfinding graph.

- **Tile Cache Obstacles:** Using DetourTileCache, we can add temporary cylindrical obstacles to the navmesh at the position of hostile mobs.²⁰
- **Re-pathing:** Once obstacles are added, the navmesh tile is locally regenerated (a fast operation). The bot then requests a new path. The A* algorithm will now see the area around the mob as unwalkable (or very high cost if using "soft" avoidance via polygon flags) and find a route around it.

6.3 Raycasting and TraceLine

To determine if an object is truly an obstacle or if a mob can see the bot, we utilize the game's internal Raycasting function, commonly known as TraceLine.

- **Functionality:** It checks for intersection between a line segment and the world geometry (WMO/M2/Terrain).
- **Implementation:** The bot defines a function pointer to the TraceLine address (typically around 0x007A3B70 in 3.3.5a, though exact offsets vary by binary).
- **Usage:**
 - **Line of Sight (LoS):** Before attacking a target, call TraceLine(PlayerPos, TargetPos). If it hits geometry, the target is obstructed.
 - **Path Validation:** Occasionally verify that the next few waypoints are visible to ensure the navmesh isn't guiding the bot through a recently spawned dynamic wall.

7. Human-Like Movement Emulation: Defeating Heuristics

Server-side anti-cheat mechanisms (like the "Anticheat" modules in TrinityCore) monitor movement patterns for anomalies. Standard bot movement is characterized by:

1. **Linearity:** Perfect straight lines between waypoints.
2. **Instant Angular Velocity:** Instantly changing facing direction (0 to 180 degrees in 1 frame).
3. **Perfect Precision:** Always clicking/moving to the exact float coordinate of a waypoint.

To survive, the bot must emulate the imperfections of human motor control.

7.1 Spline Interpolation: Smoothing the Path

The path returned by Detour is a `std::vector<Vector3>` of polygon centroids or vertices—a jagged polygonal chain. We must transform this into a smooth curve.

Catmull-Rom Splines: Catmull-Rom splines are superior to simple Bezier curves for this application because they are guaranteed to pass *through* the control points (the path nodes). A Bezier curve approximates the points, which might cause the bot to clip through a wall if the control point is near a corner.²¹

The spline $P(t)$ between points P_1 and P_2 (using previous point P_0 and next point P_3) is calculated as:

$$P(t) = 0.5 \times ((2P_1) + (-P_0 + P_2)t + (2P_0 - 5P_1 + 4P_2 - P_3)t^2 + (-P_0 + 3P_1 - 3P_2 + P_3)t^3)$$

where t ranges from 0 to 1.

By sampling points along this spline, the bot moves in a fluid, continuous arc rather than a series of straight lines.

Bezier Curves for Corners:

Specifically for sharp turns, a Quadratic Bezier curve can be used to "cut the corner," simulating a player optimizing their turn radius. We calculate a point shortly before the corner (A) and shortly after (B), using the corner waypoint (W) as the control point. The bot traverses the curve from A to B , effectively smoothing the sharp vertex.

7.2 Micro-Deviations via Perlin Noise

Humans rarely run in a mathematically perfect line. We drift slightly due to camera adjustments or input imprecision.

- **Implementation:** We overlay a 1D Perlin noise function onto the cross-track error of the

movement vector.

- **Effect:** The bot "wobbles" slightly along the path. The amplitude should be small (e.g., 0.1 - 0.3 yards) to maintain valid navigation but sufficiently random to defeat statistical linearity checks.²³

7.3 Mouse Turning Simulation

Bots often write directly to the Facing field in memory, causing the character to snap to the target angle instantly. This is a massive red flag.

- **Human Simulation:** Real players use the mouse to turn (Right-Click + Drag). This implies a continuous change in angle over time, governed by Fitts's Law.
- **Packet Spoofing:** Instead of sending one packet with the new facing, the bot should iterate over a short period (e.g., 200ms) and send intermediate CMSG_MOVE_SET_FACING packets.
- **Logic:**

$$Angle_{current} = Lerp(Angle_{start}, Angle_{target}, EaseOut(t))$$

Using an "Ease-Out" interpolation function mimics the physical motion of a mouse flick, which starts fast and slows down as it reaches the target.²⁵

7.4 Click-To-Move (CTM) vs. WASD Emulation

WoW supports two movement modes.

1. **Click-To-Move (CTM):** The player clicks a point, and the client pathfinds locally.
2. **WASD:** The player holds keys, and the client sends START_FORWARD, STOP, etc.

The Hybrid Approach:

Using CTM is easier for bots but distinct in packet logs (sending destination coordinates). To mimic WASD while using CTM logic:

- Calculate the desired heading.
- Project a point 5-10 yards ahead.
- Send CMSG_MOVE_START_FORWARD packets.
- Monitor position. When near the waypoint, send CMSG_MOVE_STOP or CMSG_MOVE_SET_FACING to turn, then START_FORWARD again.
- This mimics a player holding 'W' and turning with the mouse, generating a packet stream consistent with keyboard movement.²⁷

8. Server-Side Detection Mechanisms and Evasion

To build an undetectable bot, one must think like the server developer. We analyze the validation logic used in TrinityCore (the most common 3.3.5a core).

8.1 MovementHandler.cpp Analysis

The file MovementHandler.cpp in TrinityCore/AzerothCore contains the primary validation logic.²⁸

- **Timestamp Validation:** The server compares the client's packet timestamp with the server's expected time. If $\text{PacketTime} > \text{ServerTime} + \text{LatencyTolerance}$, it flags lag or speed hacking. **Countermeasure:** The bot must synchronize its internal clock with the server's time (via `SMSG_LOGIN_SET_TIME_SPEED`) and increment its packet timestamps accurately based on the elapsed real-time.
- **Overspeed Check:** The server calculates the distance between the last known position and the new position in the packet.

$$\text{Speed} = \frac{\text{Distance}(P_{\text{new}}, P_{\text{old}})}{T_{\text{new}} - T_{\text{old}}}$$

If $\text{Speed} > \text{UnitSpeed} \times \text{Tolerance}$, the server flags a cheat. **Countermeasure:** The bot must respect the character's current movement speed modifiers (auras, snares) and never move faster than allowed physics. Navmesh movement implicitly respects this by following valid geometry.

- **Z-Axis/Gravity:** The server checks if the player is in the air. If `IsFlying` is false but the Z-coordinate increases (without a jump flag), it flags a "Fly Hack" or "Climb Hack". **Countermeasure:** Recast's walkableClimb parameter ensures the path never ascends vertical surfaces greater than a stair step.

8.2 Warden (Client-Side Protection)

Warden is Blizzard's anti-cheat module that runs within the client. It scans for:

- **Memory Hooks:** Checks if the first bytes of API functions (e.g., `EndScene`, `Lua_DoString`) have been overwritten with JMP instructions (Detours).
- **Module Scanning:** Iterates `GetModuleHandle` or the PEB to find unauthorized DLLs.
- **Page Protections:** Checks if code sections are marked `PAGE_EXECUTE_READWRITE`.

Evasion Strategies:

- **VMT Hooking:** Instead of detouring the function start (inline hook), swap the pointer in the Interface's Virtual Method Table. Warden checks VMTs less frequently (or not at all in 3.3.5a era configs).
- **Manual Mapping:** As discussed in Section 2.1, this hides the DLL from module lists.
- **Polymorphism:** Recompiling the bot with different optimization flags or code scrambling to change the byte signature of the detection routines.

9. Conclusion

The creation of a robust bot framework for WoW 3.3.5a is a feat of software engineering that

demands rigour. By leveraging the **Recast/Detour** library, the bot utilizes the same navigational "brain" as the server, ensuring geometric consistency. The implementation of **Texture-Based Area Costs** elevates the pathfinding from simple graph traversal to intelligent, road-preferring routing that mimics human intuition.

Furthermore, the integration of **Spline Interpolation** and **Perlin Noise** into the movement controller transforms robotic vectors into organic, fluid motion, neutralizing heuristic analysis. When combined with a **Manual Mapped DLL** architecture and synchronized **EndScene Hooking**, the result is a system that is not only performant but also highly resistant to both automated and manual detection. The path to undetectability lies not in hiding the bot, but in perfecting its simulation of humanity.

Table 1: Navmesh Generation Parameters (WoW 3.3.5a)

Parameter	Value	Description
cs (Cell Size)	0.33 yards	Resolution of the voxel grid.
ch (Cell Height)	0.33 yards	Vertical resolution.
walkableSlopeAngle	50.0 degrees	Max slope a character can walk up.
walkableHeight	2.0 yards	Minimum ceiling height.
walkableClimb	0.5 yards	Max step height (stairs/curbs).
walkableRadius	0.6 yards	Character collision radius.
minRegionArea	20	Filters out small disconnected islands.

Table 2: Area Cost Weights for Road Preference

Area Type	Area ID	Weight	Effect on Pathfinding
Road	1	1.0	Preferred route.

Ground	2	4.0	Valid but penalized (4x cost).
Grass	3	4.0	Treated same as ground.
Water	4	20.0	Avoid at all costs unless no other option.
Deep Water	5	100.0/Infinity	Non-walkable.

References: ¹

Works cited

1. Calling function in injected DLL - c++ - Stack Overflow, accessed February 19, 2026, <https://stackoverflow.com/questions/10057687/calling-function-in-injected-dll>
2. Calling a function in an injected DLL? - Stack Overflow, accessed February 19, 2026, <https://stackoverflow.com/questions/13428881/calling-a-function-in-an-injected-dll>
3. 001. Recast Detour Prototype - Pathfinding with Navigation Meshes - YouTube, accessed February 19, 2026, <https://www.youtube.com/watch?v=ON7VFbJmRUk>
4. ADT File - ClientFiles - getMaNGOS | The home of MaNGOS, accessed February 19, 2026, <https://www.getmangos.eu/wiki/referenceinfo/clientfiles/adt-file-r20028/>
5. wow_alchemy_adt - Rust - Docs.rs, accessed February 19, 2026, <https://docs.rs/wow-alchemy-adt>
6. ADT/v18 - wowdev, accessed February 19, 2026, <https://wowdev.wiki/ADT/v18>
7. FileDataID - Wowpedia - Your wiki guide to the World of Warcraft - Fandom, accessed February 19, 2026, <https://wowpedia.fandom.com/wiki/FileDataID>
8. Linux Server Setup | TrinityCore MMo Project Wiki, accessed February 19, 2026, <https://trinitycore.info/en/install/Server-Setup/Linux-Server-Setup>
9. WMO - wowdev, accessed February 19, 2026, <https://wowdev.wiki/WMO>
10. stoneharry/mmmaps-for-custom-maps: Documentation regarding generating and debugging movement map (pathfinding) data for custom maps in WoW. - GitHub, accessed February 19, 2026, <https://github.com/stoneharry/mmmaps-for-custom-maps>
11. WoW data extraction resource sheet - Steak's Docs, accessed February 19, 2026, <https://thunderysteak.github.io/wow-data-extract-cheat-sheet>
12. mishalzaman/Irrlicht-pathfinding-recast-detour - GitHub, accessed February 19,

- 2026, <https://github.com/mishalzaman/lrrlicht-pathfinding-recast-detour>
13. Recast / Detour - Bloog Bot - drewkestell.us, accessed February 19, 2026, <https://drewkestell.us/Article/6/Chapter/20>
 14. recastnavigation/recastnavigation: Industry-standard navigation-mesh toolset for games - GitHub, accessed February 19, 2026, <https://github.com/recastnavigation/recastnavigation>
 15. Integrating recast & detour to project #583 - GitHub, accessed February 19, 2026, <https://github.com/recastnavigation/recastnavigation/discussions/583>
 16. Detour - Recast Navigation, accessed February 19, 2026, https://recastnav.com/group_detour.html
 17. Prioritize roads when pathfinding? - Support Forum - Arongranberg.com, accessed February 19, 2026, <https://forum.arongranberg.com/t/prioritize-roads-when-pathfinding/4117>
 18. Navigation Areas and Costs | AI Navigation | 2.0.10 - Unity - Manual, accessed February 19, 2026, <https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/AreasAndCosts.html>
 19. Wow Classic TBC: A Journey into Bot Creation | by lconstan - Medium, accessed February 19, 2026, <https://medium.com/@lconstan/wow-classic-tbc-a-journey-into-bot-creation-1ae9f1e54db9>
 20. Recast/Detour dynamic objects · Issue #457 - GitHub, accessed February 19, 2026, <https://github.com/recastnavigation/recastnavigation/issues/457>
 21. cheginit/catsmoothing: Smoothing Shapely Geometries with Catmull-Rom Splines - GitHub, accessed February 19, 2026, <https://github.com/cheginit/catsmoothing>
 22. Smoothing 3d geometry, like a tunnel, with Catmull-Rom splines - Musing Mortoray, accessed February 19, 2026, <https://mortoray.com/smoothing-3d-geometry-like-a-tunnel-with-catmull-rom-splines/>
 23. DESIGNING EFFECTIVE COMMUNICATION STRATEGIES FOR HUMAN-ROBOT COLLABORATION by Allison V. Sauppé A dissertation submitted in par - University of Wisconsin-Madison, accessed February 19, 2026, <https://asset.library.wisc.edu/1711.dl/T3THBOQCYAMAL8P/R/file-66aef.pdf>
 24. What is the application of interpolation formula? - Quora, accessed February 19, 2026, <https://www.quora.com/What-is-the-application-of-interpolation-formula>
 25. Emulating Human-Like Mouse Movement Using Bezier Curves and Behavioural Models for Advanced Web Automation - IJIRT, accessed February 19, 2026, https://ijirt.org/publishedpaper/IJIRT183343_PAPER.pdf
 26. sarperavci/human_mouse: Ultra-realistic human mouse movements using bezier curves and spline interpolation. Natural cursor automation. - GitHub, accessed February 19, 2026, https://github.com/sarperavci/human_mouse
 27. Movement Packets - WoW Classic General Discussion - World of Warcraft Forums, accessed February 19, 2026, <https://us.forums.blizzard.com/en/wow/t/movement-packets/228847>
 28. TrinityCore/src/server/game/Handlers/MovementHandler.cpp at master - GitHub,

- accessed February 19, 2026,
<https://github.com/TrinityCore/TrinityCore/blob/master/src/server/game/Handlers/MovementHandler.cpp>
29. azerothcore-wotlk/src/server/game/Entities/Object/Object.cpp at master - GitHub, accessed February 19, 2026,
<https://github.com/azerothcore/azerothcore-wotlk/blob/master/src/server/game/Entities/Object/Object.cpp>
 30. AzDeltaQQ/WotLKRotations: A Python-based experimental framework for interacting with World of Warcraft (3.3.5a - 12340 client) memory to monitor game state and potentially execute combat rotations. (Extremely Work-in-progress) - GitHub, accessed February 19, 2026,
<https://github.com/AzDeltaQQ/WotLKRotations>
 31. trickerer/AzerothCore-wotlk-with-NPCBots - GitHub, accessed February 19, 2026, <https://github.com/trickerer/AzerothCore-wotlk-with-NPCBots>
 32. Click To Move - WowDev wiki, accessed February 19, 2026,
https://wowdev.wiki/Click_To_Move
 33. SpeedHack · Issue #21 · azerothcore/mod-anticheat - GitHub, accessed February 19, 2026, <https://github.com/azerothcore/mod-anticheat/issues/21>
 34. fzippp/catmullrom: A Catmull-Rom spline implementation. - GitHub, accessed February 19, 2026, <https://github.com/fzippp/catmullrom>
 35. Windows Server Setup | TrinityCore MMo Project Wiki, accessed February 19, 2026, <https://trinitycore.info/en/install/Server-Setup/Windows-Server-Setup>
 36. WoWObjectManager/WoWObjectManager/Offsets.cs at master · Xartrick/WoWObjectManager - GitHub, accessed February 19, 2026,
<https://github.com/Xartrick/WoWObjectManager/blob/master/WoWObjectManager/Offsets.cs>
 37. WoW-Object-Manager/WoWObjMgr/PlayerScan.cs at master - GitHub, accessed February 19, 2026,
<https://github.com/johnmoore/WoW-Object-Manager/blob/master/WoWObjMgr/PlayerScan.cs>
 38. TrinityCore/src/server/game/AntiCheat/AntiCheat.h at master · Elevim/TrinityCore - GitHub, accessed February 19, 2026,
<https://github.com/Elevim/TrinityCore/blob/master/src/server/game/AntiCheat/AntiCheat.h>
 39. Anti-Cheat, An Analysis | Games - Maddy Miller, accessed February 19, 2026,
<https://madelinemiller.dev/blog/anticheat-an-analysis/>
 40. how does WoW entities pathfinding? : r/gamedev - Reddit, accessed February 19, 2026,
https://www.reddit.com/r/gamedev/comments/1qwbct7/how_does_wow_entities_pathfinding/